

AGENDA WEEK 4

Day 1:

- FUNCTIONS REVIEW
- PSEUDOCODE

Day 2:

- STRING METHODS (briefly)
- INDEXING AND SLICES

0. FUNCTION REVIEW

REVIEW: ANATOMY OF A FUNCTION DEFINITION

the `def` keyword: lets python know
you're defining a function



`def` *name (parameters):*
 # statements to execute

REVIEW: ANATOMY OF A FUNCTION DEFINITION

whatever name you give
to the function



```
def name (parameters):  
    # statements to execute
```

REVIEW: ANATOMY OF A FUNCTION DEFINITION

this can be pretty much
anything (it can't be
"for", "in", "def", or any
other keyword)



```
def name (parameters):  
    # statements to execute
```

REVIEW: ANATOMY OF A FUNCTION DEFINITION

this probably makes more sense
than it did the first time you saw
this slide



```
def name (parameters):  
    # statements to execute
```

REVIEW: ANATOMY OF A FUNCTION DEFINITION

```
def name (parameters):  
    # statements to execute
```



Don't forget the colon!

ANATOMY OF A FUNCTION DEFINITION

`def name (parameters):`

`# statements to execute`



The stuff the function
does when called

EXAMPLE:

```
def annoyingFunction(repetitions):  
    for word in range(repetitions):  
        print("hello!")
```



the value of the repetitions variable
is whatever value given as an
argument in the function call

EXAMPLE:

```
def annoyingFunction(repetitions, word):  
    for word in range(repetitions):  
        print(word)
```

```
main():  
    annoyingFunction(5, "hello!")
```

so if we call it this way, we print
hello 5 times



RETURN VALUES:

```
def annoyingFunction(repetitions):  
    output = ""  
    for word in range(repetitions):  
        output = output + "hello"  
    return output
```

```
main():  
    annoyingFunction(5)
```



this doesn't do anything, because
we don't do anything with the string the
function returns

RETURN VALUES:

```
def annoyingFunction(repetitions):  
    output = ""  
    for word in range(repetitions):  
        output = output + "hello"  
    return output
```

```
main():  
    result = annoyingFunction(5)  
    print(result)
```



prints "hellohellohellohellohello"

RETURN VALUES:

```
def annoyingFunction(repetitions):  
    output = ""  
    for word in range(repetitions):  
        output = output + "hello"  
    return output
```

```
main():  
    print(annoyingFunction(5))
```



prints "hellohellohellohellohello", as in previous slide

RETURN VALUES:

```
def annoyingFunction(repetitions):  
    output = ""  
    for word in range(repetitions):  
        output = output + "hello"  
    return output  
print(output)
```



once we return from a function, we stop running that function. this print statement never runs, because it comes after the return keyword

```
main():  
    annoyingFunction(5)
```

RETURN VALUES:

```
def isUpper(userInput):  
    if userInput.isupper():  
        print("it's uppercase!")  
        return True  
    else:  
        print("it's not uppercase")  
        return False
```



this stuff comes after a return statement, but it doesn't come after a return statement is actually executed – we only do the stuff after this else statement if the original if statement wasn't true.

1. PSEUDOCODE

PSEUDOCODE

- Recall that an algorithm is a step-by-step description of a process
- Pseudocode is a way to describe algorithms in an informal mixture of english and code.
- Writing pseudocode before writing working code is a strategy to speed up software development
- How does writing pseudocode speed up development? It lets you figure out the top-level structure of your program before you have to think about syntax detail.
- Pseudocode doesn't have to be particularly precise.
- Can be a good way to figure out what functions you need to define

PSEUDOCODE EXAMPLE – waking up

pseudocode for morning routine (if you're me)

waitForAlarm()

if not monday throught thursday:

 turn off alarm

else:

 pushSnooze(randrange(2, 5))

get out of bed

go to bathroom

take shower

get dressed

drinkCupsOfCoffee(randrange(2, 5))

bikeToBart

bartToBus

goToOffice

if notEnoughCoffee:

 getFromFacultyLounge

IN-CLASS EXERCISE 1

WRITING PSEUDOCODE

Write pseudocode (NOT a full program) to ask the user for a new password. Your pseudocode, if implemented, would have to:

ask the user for a string, ask the user to re-enter the string, verify that the string is at least eight characters long, verify that that string contains at least one number, at least one lowercase letter, at least one piece of punctuation, and at least one uppercase letter.

Start over if either the two strings don't match, or if the string is too short or doesn't have all the required characters.

STRING MANIPULATION REFERENCE

- [Here](#) you can find a reference of commands you can run to do things with strings
- Some of those commands can be very useful for *implementing* the pseudocode you write, but they're not necessary for *designing* the pseudocode
- Remember, pseudocode's not Python! it's not any programming language at all!
- Instead, it's a tool you use to think through algorithm design

ONE STRATEGY FOR DECIDING WHICH FUNCTIONS TO WRITE

- Write out your `main()` function as pseudocode. Include function calls to functions that "magically" do whatever complicated thing you need to do to actually implement the code
- Go back and actually write the functions
- This is called "top-down design" – it's a good way to make sure your code is clean and readable

2. STRING MANIPULATION

STRING METHODS

- See textbook for a reference of string methods:
- try out: find, replace, strip

```
myString = " hello world. "
```

```
print(myString.strip()) → prints "hello world"
```

```
myString = myString.strip()
```

```
print(myString.find('llo')) → prints 2
```

```
print(myString.find('goodbye')) → prints -1
```

```
print(myString.replace('world' 'operator please give me number 9')) → prints "hello operator  
please give me number 9"
```

SLICE NOTATION

We've seen a couple of ways to get characters from strings by index:

```
myString = "hello"
```

```
print(myString[0])
```



prints 'h'

SLICE NOTATION

We've seen a couple of ways to get characters from strings by index:

```
myString = "hello"  
print(myString[0])
```

```
print(myString[-1])
```



prints 'o'

SLICE NOTATION

We can also print a range of characters, using *slice notation*:

```
myString = "hello world"  
print(myString[6:11])
```



prints 'world'

FUN WITH RANGES

Consider this for loop:

```
myString = "hello world"  
for x in range(len(myString)):  
    print(x)
```

what does this print?

FUN WITH RANGES

Consider this for loop:

```
myString = "hello world"  
for x in range(len(myString)):  
    print(x)
```

what does this print?

answer: 0
1
2
3
4
5
6
7
8
9
10

FUN WITH RANGES

Consider this for loop:

```
myString = "hello world"  
for x in range(len(myString)):  
    print(x)
```

this may not seem terribly useful, but notice that these are the indices of all the characters in myString

```
myString[0] → h  
myString[4] → o  
myString[10] → d
```

answer: 0
1
2
3
4
5
6
7
8
9
10

FUN WITH RANGES

Consider this for loop:

```
myString = "hello world"  
for x in range(len(myString)):  
    print(myString[x])
```



(remember, the len() function prints the length of the string you pass in)

okay, what what does *this* print?

FUN WITH RANGES

Consider this for loop:

```
myString = "hello world"  
for x in range(len(myString)):  
    print(myString[x])
```



(remember, the len() function prints the length of the string you pass in)

okay, what what does *this* print?

answer: h
e
l
l
o

w
o
r
l
d

FUN WITH ACCUMULATORS

Consider this for loop:

```
myString = "hello world"
```

```
outputString = ""
```

```
for x in range(len(myString)):
```

```
    outputString = outputString + myString[x]
```

okay, what what does *this* print?

answer: hello world

IN-CLASS DISCUSSION

Remember that the range function is pretty flexible.

`range(starting point, ending point, increment)`

```
for x in range(0, 10, 2):  
    print(x)
```

prints:

0
2
4
6
8

IN-CLASS DISCUSSION

Remember that the range function is pretty flexible.

`range(starting point, ending point, increment)`

```
for x in range(10, -1, -2):  
    print(x)
```

prints:

10

8

6

4

2

IN-CLASS DISCUSSION

What does this print?

```
myString = "hello world"  
output = "  
for x in range(len(myString)-1, -1, -1):  
    output = output + myString[x]  
print(output)
```

answer:
dlrow olleh

IN-CLASS EXERCISE 2

Write a function that takes a string, then returns a new string consisting of the original string plus the original string backwards.

`print(makePalindrome('hello'))` → prints `helloolleh`

`print(makePalindrome('dracula'))` → prints `draculaalucard`

FOR REFERENCE:

```
myString = "hello world"
```

```
output = ""
```

```
for x in range(len(myString)-1, -1, -1):
```

```
    output = output + myString[x]
```

```
print(output)
```